

Event Ticketing Platform

Complete Project Design & Architecture Document

A real-time, high-concurrency seat-based ticket booking system

1. Project Overview

This project is an Event Ticketing Platform that simulates the exact scenario that breaks most naive booking systems: thousands of people trying to buy a limited number of seats for a popular event at the same moment. On the surface it looks like a simple e-commerce flow — browse events, pick seats, pay, receive a ticket — but underneath it solves a genuine distributed-systems problem: guaranteeing that a seat is never sold to two people at once, while handling a traffic spike gracefully and giving every user clear, real-time feedback.

The platform supports two primary roles — Customers, who browse events and purchase seats, and Organizers, who create events, define seat maps, and monitor live sales. The system is built around three core engineering challenges: concurrency control (never overselling a seat), fairness under load (a virtual waiting room for high-demand events), and real-time synchronization (every viewer sees seat status change live).

The value of this project is that its core problem — preventing double-booking under concurrency — is provably testable: a load test simulating hundreds of concurrent buyers against a small number of seats should show that exactly the right number of purchases succeed, no more. That result is demonstrable evidence of correct system design, not just working features.

2. Complete Tech Stack

Frontend

- Next.js (React framework) — hosted on Vercel
- WebSocket client (Socket.IO) for live seat map and waiting room updates
- Stripe.js / Stripe Elements for secure client-side payment collection

Backend

- Node.js with Express (API server)
- Socket.IO server for real-time broadcast (seat status, waiting room position)
- JWT-based authentication with role embedded in the token (customer / organizer)

Data Layer

- PostgreSQL — source of truth for all durable/permanent data (events, seats, orders, tickets, payments)
- Redis — ephemeral, fast-moving state: seat holds (TTL locks), waiting room queue, short-lived caches

Payments

- Stripe (test mode) — PaymentIntents API plus webhook handling for asynchronous payment confirmation

Hosting / Infrastructure (free tier)

- Neon — serverless PostgreSQL hosting
- Upstash — serverless Redis hosting
- Railway or Render — backend API hosting
- Vercel — frontend hosting

Testing / Verification

- k6 or autocannon — load testing tool used to simulate concurrent buyers and prove the system never oversells a seat

3. User Roles & Dashboards

A. Customer Dashboard

- Browse and search events, filter by date, location, category
- View event details (venue, time, price range, seat map preview)
- Join the waiting room when an event is under high demand
- View a live seat map and select seats
- Checkout with Stripe test-mode payment
- View “My Tickets” — purchased tickets with QR codes
- View order history

B. Organizer Dashboard

- Create events and assign them to a venue
- Define pricing per seat section
- Set the ticket sale window (start/end), which determines when the waiting room can activate
- View a live sales dashboard — seats sold, revenue, remaining inventory
- View waiting room statistics — queue size, admission rate
- Manage event status: draft → published → live → sold_out → closed

C. Super Admin Dashboard (optional, future scope)

- Approve or reject organizer accounts
- Platform-wide analytics: total events, total revenue, active users

4. Database Schema (PostgreSQL)

The schema is fully relational and normalized. Each table is designed around a specific responsibility, and foreign keys enforce data integrity across the booking lifecycle.

organizers

```
org_id          SERIAL PRIMARY KEY
org_name        VARCHAR(150) NOT NULL
org_email       VARCHAR(150) UNIQUE NOT NULL
org_phone       VARCHAR(20)
password_hash   VARCHAR(255) NOT NULL
created_at, updated_at
```

customers

```
customer_id     SERIAL PRIMARY KEY
customer_name   VARCHAR(150) NOT NULL
customer_email  VARCHAR(150) UNIQUE NOT NULL
phone_number    VARCHAR(20)
password_hash   VARCHAR(255) NOT NULL
default_location VARCHAR(150) -- default filter for event listing
created_at, updated_at
```

venues

Venues are reusable across many events. `seat_layout_json` is a template describing sections, rows, and seat counts; actual Seats rows are generated per event from this template when an event is published.

venue_id	SERIAL PRIMARY KEY
venue_name	VARCHAR(150) NOT NULL
address	VARCHAR(255)
city	VARCHAR(100)
total_capacity	INT
seat_layout_json	JSONB
created_at	

events

event_id	SERIAL PRIMARY KEY
org_id	INT REFERENCES organizers(org_id)
venue_id	INT REFERENCES venues(venue_id)
event_name	VARCHAR(200) NOT NULL
description	TEXT
category	VARCHAR(100)
event_start_time	TIMESTAMP NOT NULL
event_end_time	TIMESTAMP NOT NULL
status	VARCHAR(20) DEFAULT 'draft' -- draft / published / live / sold_out / closed
sale_window_start	TIMESTAMP -- when ticket sales open (drives waiting room)
sale_window_end	TIMESTAMP
created_at, updated_at	

seats

Auto-generated per event from the venue's `seat_layout_json` template. The `version` column implements optimistic locking — it is checked and incremented on the final sale write to guarantee a seat can never be sold twice, even under concurrent requests.

seat_id	SERIAL PRIMARY KEY
event_id	INT REFERENCES events(event_id)
section	VARCHAR(50)
row_label	VARCHAR(10)
seat_number	INT
price	DECIMAL(10,2) NOT NULL
status	VARCHAR(20) DEFAULT 'available' -- available / locked / sold
version	INT DEFAULT 0
created_at	
UNIQUE (event_id, section, row_label, seat_number)	

orders

order_id	SERIAL PRIMARY KEY
customer_id	INT REFERENCES customers(customer_id)
event_id	INT REFERENCES events(event_id)
status	VARCHAR(20) DEFAULT 'pending' -- pending / confirmed / failed / expired
total_amount	DECIMAL(10,2) NOT NULL
idempotency_key	VARCHAR(100) UNIQUE NOT NULL
created_at, updated_at	

tickets

One row per purchased seat, kept separate from orders so each ticket can be individually checked in, scanned, or cancelled.

ticket_id	SERIAL PRIMARY KEY
order_id	INT REFERENCES orders(order_id)

```

seat_id      INT REFERENCES seats(seat_id)
price        DECIMAL(10,2) NOT NULL
qr_code      VARCHAR(255) UNIQUE
checked_in   BOOLEAN DEFAULT FALSE
created_at

```

payments

Kept separate from orders because a single order may have multiple or failed payment attempts before succeeding — required for correct webhook idempotency handling.

```

payment_id    SERIAL PRIMARY KEY
order_id      INT REFERENCES orders(order_id)
provider      VARCHAR(50) DEFAULT 'stripe'
provider_payment_id VARCHAR(150)
amount        DECIMAL(10,2) NOT NULL
status        VARCHAR(20) DEFAULT 'pending'
              -- pending / succeeded / failed / refunded
created_at

```

5. Redis Key Structures

```

seat:{seat_id}
  value: customer_id who holds it
  TTL: 300 seconds
  purpose: temporary seat hold during checkout

waitingroom:{event_id}  (sorted set)
  members: customer_id
  score: timestamp of joining the queue
  purpose: FIFO-ordered admission during high-demand sales

active_users:{event_id}  (set)
  members: customer_id currently in seat selection
  purpose: compared against a threshold to decide waiting room activation

event_status:{event_id}  (string, short TTL cache)
  value: upcoming / live / closed
  purpose: avoids recomputing event live-status on every request

events_list:{location}  (string, short TTL cache)
  value: cached JSON of event listing results

```

6. Concurrency & Locking Strategy

The system uses a hybrid locking strategy rather than relying on a single technique, because different phases of the booking flow have different requirements.

Phase 1 — Seat Selection (Redis Distributed Lock)

The moment a user clicks a seat, the backend attempts an atomic Redis operation: `SET seat:{id} {customer_id} NX EX 300`. The `NX` flag guarantees only one request can successfully acquire the key if two users click the same seat simultaneously; the losing request is rejected instantly, with no waiting or blocking. The `EX` flag gives the lock a natural 5-minute expiry with no manual cleanup job required — if the user abandons checkout, Redis releases the seat automatically.

Conceptually this is a pessimistic locking strategy — the resource is reserved upfront, before anyone else can act on it — but implemented as a distributed lock rather than a classic database transaction lock. This avoids the major weakness of traditional pessimistic locking (an open database transaction blocking other queries for the entire multi-minute checkout window).

Phase 2 — Final Payment Confirmation (Optimistic Locking)

When payment succeeds, the seat's final state is written permanently to PostgreSQL using an optimistic locking check against the version column:

```
UPDATE seats
SET status = 'sold', version = version + 1
WHERE seat_id = ? AND version = ? AND status != 'sold';
```

If zero rows are updated, something has gone wrong upstream of the Redis lock (a rare edge case such as a Redis failure or duplicate webhook), and the system safely rejects the write rather than allowing an oversell. This makes PostgreSQL the ultimate, non-negotiable source of truth, independent of whether the Redis layer functioned correctly.

Why a Hybrid Approach

- Pure pessimistic (database-level) locking would hold a transaction open for the entire checkout window, blocking other users' queries and tying up database connections.
- Pure optimistic locking alone provides no protection during the browsing/deciding phase — a user could fill in payment details for minutes only to lose the seat to someone faster at the final step.
- The hybrid approach gives the fairness of first-come-first-served (via the Redis lock) with the safety and scalability of optimistic locking (via the version check) at the point of permanent write.

7. Waiting Room Logic

The waiting room does not activate for every event by default. It activates dynamically, based on real concurrency, once an event enters its active sale window (current time between `sale_window_start` and `sale_window_end`).

- Each event has a maximum concurrency threshold — the number of users allowed into active seat selection at once.
- The current count of active seat-selectors is tracked in Redis (`active_users:{event_id}`).
- If a new user arrives and the count is below the threshold, they are admitted directly.
- If the threshold is met, the user is placed into a Redis sorted set (`waitingroom:{event_id}`), scored by arrival timestamp, guaranteeing FIFO order.
- A background process periodically checks for free capacity and admits the next users in queue order, notifying them via WebSocket.

This threshold-based design mirrors how real-world ticketing platforms (e.g. queueing systems used by Ticketmaster) protect their backend from traffic spikes while remaining invisible to users during low-demand events.

Efficient “Is the Event Live” Checking

Event live-status is never computed by polling the database continuously. Instead, the status (upcoming / live / closed) is computed from `event_start_time` and `sale_window` fields once, then cached in Redis with a short TTL (around 10 seconds). Every subsequent request within that window reads the cached value directly, avoiding repeated database load while accepting a few seconds of harmless staleness in exactly when the waiting room activates.

8. Complete User & System Flow

The following traces every stage of the booking journey from landing on the website to holding a confirmed ticket, along with what happens on the backend at each step.

Stage 1 — Landing Page

The frontend requests the event listing. The backend checks a short-lived Redis cache for the event list; on a cache miss it queries PostgreSQL, filtered to events whose status is published, live, or `sold_out` and whose `event_end_time` has not yet passed, then re-caches the result.

Stage 2 — Event Details

Clicking an event fetches its full details and venue information from PostgreSQL, along with a cached live-status computation (upcoming / live / closed) that determines whether a countdown, a “Buy Tickets” button, or a “Sold Out” message is shown.

Stage 3 — Authentication & Access Check

If the user is not logged in, they are routed to login/signup, issuing a JWT on success. Once authenticated, the backend checks current active-user count for the event against its threshold to decide whether the user is admitted directly to seat selection or placed into the waiting room.

Stage 4 — Waiting Room (if applicable)

Queued users receive live position updates via WebSocket. A background process admits users from the front of the Redis sorted-set queue as capacity frees up, moving them into the active-users set and pushing an admission notification.

Stage 5 — Seat Selection

The seat map is built by combining PostgreSQL (permanent seat status) with Redis (temporary holds), so a seat that is technically still ‘available’ in Postgres but currently locked in Redis correctly displays as held. Selecting a seat attempts the Redis NX/EX lock described in Section 6.

Stage 6 — Checkout & Payment

An order is created in PostgreSQL with a pending status and a unique idempotency key, and a Stripe PaymentIntent is generated. The frontend completes payment directly with Stripe using its client secret.

Stage 7 — Stripe Webhook & Final Confirmation

Stripe calls the backend's webhook endpoint asynchronously. After verifying the webhook signature and checking idempotency, the backend performs the final optimistic-locking write to PostgreSQL, generates the ticket record and QR code, marks the order confirmed, deletes the now-unnecessary Redis lock, and broadcasts the seat's new sold status to every connected viewer via WebSocket.

Stage 8 — Confirmation

The customer is shown their confirmed order and ticket, retrievable at any time from PostgreSQL via their order history.

9. API & System Touchpoint Summary

User Action	API Endpoint	Database	Redis	WebSocket
Open homepage	GET /api/events	Postgres (cached)	Read cache	No
View event	GET /api/events/:id	Postgres	Read status cache	No
Click Buy Tickets	GET /api/events/:id/access-check	—	Read/write active users	No
Waiting room	GET /api/events/:id/waiting-room/status	—	Read sorted set	Yes
Load seat map	GET /api/events/:id/seats	Postgres	Read locks	Yes (subscribe)
Select a seat	POST /api/seats/:id/lock	Postgres (flag)	SET NX EX	Yes (broadcast)
Submit order	POST /api/orders	Postgres (insert)	Verify lock	No
Stripe webhook	POST /api/webhooks/stripe	Postgres (transaction)	Delete lock	Yes (broadcast)
View confirmation	GET /api/orders/:id	Postgres	—	No

10. Deployment Plan

Layer	Provider	Notes
Frontend	Vercel	Free tier, native Next.js support
Backend API	Railway / Render	WebSocket-capable, free/hobby tier
PostgreSQL	Neon	Serverless, scales to zero when idle
Redis	Upstash	Serverless, TTL-friendly, free tier
Payments	Stripe (test mode)	Real webhook behavior without real charges

11. Recommended Build Order

- 1. Project skeleton (backend + frontend structure, environment setup)
- 2. PostgreSQL schema as migration files, connected to Neon
- 3. Seat generation logic (venue template → per-event seat rows)
- 4. Redis seat-locking logic (the core concurrency mechanism)
- 5. Waiting room queue logic
- 6. Payments and Stripe webhook handling
- 7. Real-time WebSocket layer for live seat map and queue updates
- 8. Frontend build-out across all pages
- 9. Load testing to verify zero overselling under concurrency